

2026 GEMMA 4 开发者大赛

FinSight Gemma4 技术报告

基于本地私有化部署的金融智能体系统

项目 FinSight — 本地智能金融研究工作台

模型 Gemma-4-26B-A4B-it-NVFP4

日期 2026年6月

目录

1. 项目概述

2. Gemma4 在项目中的定位

2.1 本地主模型

3. 模型选型理由

3.1 为什么选择 Gemma4

3.2 为什么选择 Gemma-4-26B-A4B-it-NVFP4

4. 总体架构设计

4.1 系统架构

4.2 Gemma4 调用路径

5. 核心代码：Gemma4 调用逻辑

5.1 默认模型配置

6. Native Function Calling 设计

6.1 为什么需要 Native Function Calling

6.2 Native Function Calling 核心示例

6.3 工具执行闭环

7. 多模态 Multimodal 处理设计

7.1 为什么需要多模态

7.2 多模态输入格式

7.3 多模态与函数调用结合

8. Gemma4 在工作流中的调用

8.1 工作流入口

8.2 模型环境注入

8.3 LLM 语义增强输出

9. Gemma4 在智能体中的作用

- 9.1 多智能体结构
- 9.2 模型切换控制
- 9.3 智能体调用价值

10. 架构上的安全与可信设计

- 10.1 不让模型直接相信自己
- 10.2 证据优先
- 10.3 多模态复核降低解析错误

11. 与传统 RAG 的区别

12. 参赛亮点总结

- 12.1 技术亮点
- 12.2 业务亮点

13. 结论

14. 模型部署指南

- 14.1 部署环境准备
- 14.2 模型下载与配置
- 14.3 vLLM 服务启动
- 14.4 关键参数说明
- 14.5 服务验证与监控
- 14.6 与 FinSight 集成

附录：完整部署脚本

1. 项目概述

FinSight 是一个面向财报研究、投研分析、事实核查、持续跟踪和法务合规的本地智能研究工作台。项目核心目标不是简单让大模型"阅读 PDF 并生成报告", 而是构建一条可审计、可追溯、可复核的金融分析链路:

官方财报 PDF

- > PDF 解析与表格抽取
- > 页码 / 表格 / bbox / Markdown / JSON 证据层
- > Wiki 与数据库结构化资产
- > Gemma4 本地语义增强
- > 多智能体分析、核查、跟踪、法务问答
- > 前端工作台展示与人工复核

在参赛版本中, Gemma4 被设计为 FinSight 的**本地主模型**, 承担本地推理、语义增强、Native Function Calling 和多模态财报证据复核能力。Qwen3.6 保留为备用模型, 用于手动切换或特殊长上下文场景。

2. Gemma4 在项目中的定位

2.1 本地主模型

当前项目默认使用的 Gemma4 规格为:

模型名: Gemma-4-26B-A4B-it-NVFP4
服务地址: `http://127.0.0.1:8006/v1`
接口协议: OpenAI-compatible Chat Completions
部署方式: 本地 vLLM 服务

相关配置位于:

```
apps/api-gateway/services/llm_settings.py
apps/api-gateway/services/hermes_model_control.py
.env.example
README.md
```

Gemma4 在项目中承担三类核心职责:

1. 本地推理底座

- 为 workflow、设置页、智能体提供统一的本地模型入口。

2. 财报语义增强

- 对规则层抽取的事实、证据、段落和声明进行业务语义归纳。

3. 智能体工具调用与多模态复核

- 通过 Native Function Calling 调用财报查询、PDF 页图像、表格来源、法规检索等工具。
- 通过 Multimodal 输入处理 PDF 页面截图、表格截图、公告页面图像等视觉证据。

3. 模型选型理由

3.1 为什么选择 Gemma4

FinSight 的业务场景对模型有几个明确要求：

表 1 FinSight 业务场景对模型的要求

要求	说明
本地可部署	财报、合规、法律材料具有隐私和审计要求，不适合全部外发云端
中文金融文本能力	需要理解 A 股年报、财务附注、管理层讨论、法规条款
工具调用能力	需要主动调用数据库、Wiki、PDF 来源、法规检索，而不是凭记忆回答
多模态能力	需要读取 PDF 页面截图、表格图像、页码、版式、脚注和扫描内容
成本可控	财报分析链路长、token 量大，云端长期调用成本较高
可审计	输出必须能回到 PDF 页码、表格索引、原始证据和结构化数据

Gemma4 的优势在于它可以作为本地模型部署，同时兼顾文本推理、结构化输出、函数调用和多模态输入，适合构建私有化金融智能体系统。

3.2 为什么选择 `Gemma-4-26B-A4B-it-NVFP4`

本项目选择该规格，主要基于以下技术权衡：

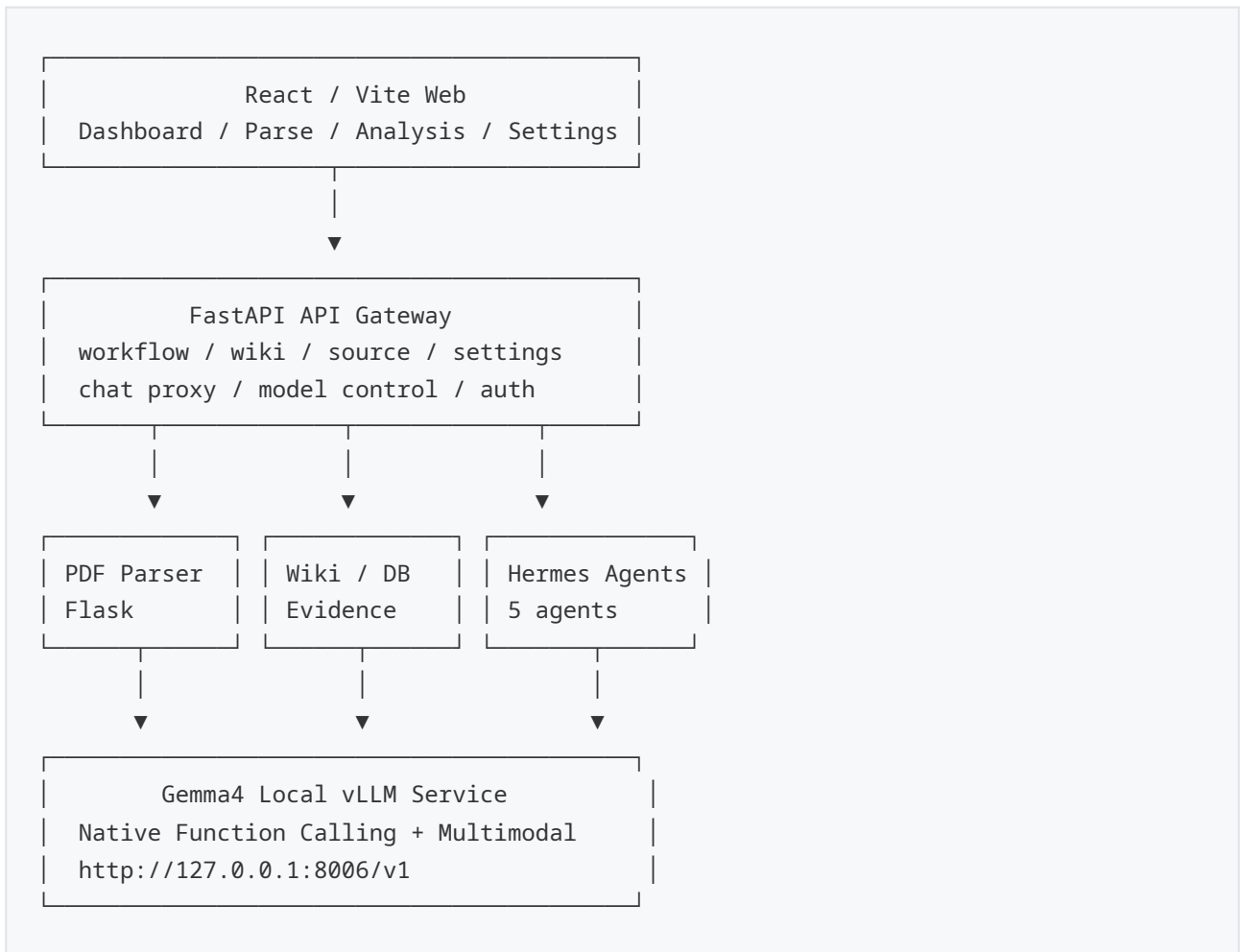
表 2 模型规格选择的技术权衡

维度	选择理由
26B 总参数规模	提供足够的语言理解和复杂金融推理能力，适合财报、法律和多智能体场景
A4B 激活规模	在推理时降低实际活跃计算量，兼顾模型能力和本地部署成本
it 指令微调版本	更适合对话、工具调用、结构化 JSON 输出和业务任务执行
NVFP4 量化规格	降低显存压力，使模型更适合在本地 GPU / vLLM 环境中运行
本地 vLLM 服务	统一暴露 OpenAI-compatible 接口，方便前后端和工作流集成
与 Qwen3.6 互补	Gemma4 作为主模型，Qwen3.6 作为备用模型，形成双本地模型冗余

换句话说，Gemma4 不是单纯追求最大参数，而是在本地可运行、成本可控、工具调用稳定、多模态可扩展之间取得平衡。

4. 总体架构设计

4.1 系统架构



4.2 Gemma4 调用路径

Gemma4 的调用主要经过三条路径：

1. 设置页模型测试

Web Settings -> /api/settings/test -> Gemma4 /chat/completions

2. workflow语义增强

/api/workflow/task/{task_id}/semantic

-> 规则语义层

-> Gemma4 LLM semantic enrichment

-> semantic/llm/<report_id>/*.json

3. 智能体本地模式

用户在 Agent 对话中要求切换本地模型

-> hermes_model_control.py

-> Gemma4 profile config

-> Hermes gateway 使用 Gemma4

5. 核心代码：Gemma4 调用逻辑

5.1 默认模型配置

项目中 Gemma4 被设置为默认本地模型：


```
# apps/api-gateway/services/llm_settings.py

LOCAL_GEMMA4_PROVIDER = {
    "enabled": True,
    "providerName": "本地 vLLM / Gemma4",
    "baseUrl": (
        os.environ.get("FINSIGHT_LOCAL_LLM_BASE_URL")
        or os.environ.get("FINSIGHT_GEMMA4_LLM_BASE_URL")
        or "http://127.0.0.1:8006/v1"
    ),
    "apiKey": os.environ.get("FINSIGHT_LOCAL_LLM_API_KEY")
        or os.environ.get("FINSIGHT_GEMMA4_LLM_API_KEY", ""),
    "model": (
        os.environ.get("FINSIGHT_LOCAL_LLM_MODEL")
        or os.environ.get("FINSIGHT_GEMMA4_LLM_MODEL")
        or "Gemma-4-26B-A4B-it-NVFP4"
    ),
    "temperature": 0.2,
    "maxTokens": 8192,
    "timeoutSeconds": 600,
    "chatTemplateKwargs": {"enable_thinking": False},
}
```

该配置的作用是：

- 设置 Gemma4 为默认 `local` provider。
- 将本地模型地址固定到 `8006`。
- 允许通过环境变量覆盖模型地址、模型名和 API Key。
- 保持 OpenAI-compatible 调用方式，便于替换模型或接入 vLLM。

6. Native Function Calling 设计

6.1 为什么需要 Native Function Calling

FinSight 不能让模型仅凭上下文生成结论。财报分析必须回答三个问题：

1. 这个结论来自哪份报告？
2. 数据在哪个 PDF 页？

3. 是否能回到表格、段落、法规条款或数据库记录？

因此，Gemma4 在项目中不是"自由聊天模型"，而是一个可以主动调用工具的智能推理核心。

Native Function Calling 使模型可以按需调用：

表 3 Native Function Calling 工具列表

工具	功能
<code>get_company_reports</code>	查询公司已有报告和 Wiki 入口
<code>get_financial_metrics</code>	查询三大表和关键财务指标
<code>get_pdf_page_image</code>	获取 PDF 指定页图像，用于多模态复核
<code>get_source_table</code>	获取表格来源、页码、Markdown 行号
<code>search_legal_rules</code>	调用 Milvus 法规检索
<code>create_review_queue</code>	生成待人工复核事项

6.2 Native Function Calling 核心示例

下面是参赛核心代码设计，用于展示 Gemma4 原生函数调用链路。

```
# apps/api-gateway/services/gemma4_native_client.py

import base64
import os
from pathlib import Path
from typing import Any

import httpx

GEMMA4_BASE_URL = os.getenv("FINSIGHT_LOCAL_LLM_BASE_URL", "http://127.0.0.1:8006/v1")
GEMMA4_MODEL = os.getenv("FINSIGHT_LOCAL_LLM_MODEL", "Gemma-4-26B-A4B-it-NVFP4")
GEMMA4_API_KEY = os.getenv("FINSIGHT_LOCAL_LLM_API_KEY", "")

GEMMA4_TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "get_financial_metrics",
            "description": "查询指定公司的三大表、关键财务指标和证据来源。",
            "parameters": {
                "type": "object",
                "properties": {
                    "company_id": {
                        "type": "string",
                        "description": "公司 ID, 例如 600309-万华化学"
                    },
                    "report_id": {
                        "type": "string",
                        "description": "报告 ID, 例如 2025-annual"
                    },
                    "metric_names": {
                        "type": "array",
                        "items": {"type": "string"},
                        "description": "需要查询的指标名称"
                    }
                }
            },
            "required": ["company_id", "report_id"]
        }
    }
]
```

6.3 工具执行闭环

```

async def run_gemma4_tool_loop(user_question: str, context: dict[str, Any]) -> str:
    messages = [
        {
            "role": "system",
            "content": (
                "你是 FinSight 财报研究助手。所有关键财务结论必须通过工具查询证据，"
                "回答中必须包含公司、报告期、PDF 页码或表格索引。"
            ),
        },
        {
            "role": "user",
            "content": user_question,
        },
    ]

    first = await call_gemma4(messages, tools=GEMMA4_TOOLS)
    assistant_message = first["choices"][0]["message"]
    messages.append(assistant_message)

    tool_calls = assistant_message.get("tool_calls") or []

    for tool_call in tool_calls:
        name = tool_call["function"]["name"]
        args = json.loads(tool_call["function"]["arguments"])

        if name == "get_financial_metrics":
            result = await get_financial_metrics(**args)
        elif name == "get_source_table":
            result = await get_source_table(**args)
        elif name == "get_pdf_page_image":
            result = await get_pdf_page_image(**args)
        elif name == "search_legal_rules":
            result = await search_legal_rules(**args)
        else:
            result = {"error": f"unknown tool: {name}"}

        messages.append(
            {
                "role": "tool",

```

```

async def run_gemma4_tool_loop(user_question: str, context: dict[str, Any]) -> str:
    messages = [
        {
            "role": "system",
            "content": (
                "你是 FinSight 财报研究助手。所有关键财务结论必须通过工具查询证据，"
                "回答中必须包含公司、报告期、PDF 页码或表格索引。"
            ),
        },
        {
            "role": "user",
            "content": user_question,
        },
    ]

    first = await call_gemma4(messages, tools=GEMMA4_TOOLS)
    assistant_message = first["choices"][0]["message"]
    messages.append(assistant_message)

    tool_calls = assistant_message.get("tool_calls") or []

    for tool_call in tool_calls:
        name = tool_call["function"]["name"]
        args = json.loads(tool_call["function"]["arguments"])

        if name == "get_financial_metrics":
            result = await get_financial_metrics(**args)
        elif name == "get_source_table":
            result = await get_source_table(**args)
        elif name == "get_pdf_page_image":
            result = await get_pdf_page_image(**args)
        elif name == "search_legal_rules":
            result = await search_legal_rules(**args)
        else:
            result = {"error": f"unknown tool: {name}"}

        messages.append(
            {
                "role": "tool",
                "tool_call_id": tool_call["id"],
                "name": name,

```

该设计体现了 Native Function Calling 的核心价值：

- 模型决定何时调用工具。
- 工具返回结构化证据。
- 模型基于工具结果生成最终回答。
- 所有证据来源可以被日志记录和人工审计。

7. 多模态 Multimodal 处理设计

7.1 为什么需要多模态

财报 PDF 中存在大量文本模型难以可靠处理的内容：

表 4 多模态处理场景对比

场景	纯文本问题	多模态价值
扫描版页面	OCR 可能漏字或错位	直接读取页面图像进行复核
复杂表格	Markdown 表格可能错列	视觉检查表头、列名、合并单元格
页眉页脚	页码和报告页可能不一致	识别真实 PDF 页与文档页
脚注	脚注和正文距离远	视觉定位脚注对应关系
图片化公告	无法直接文本解析	使用图像输入理解内容

7.2 多模态输入格式

Gemma4 通过 OpenAI-compatible multimodal message 输入 PDF 页面图像。

```

def image_to_data_url(path: str) -> str:
    image_bytes = Path(path).read_bytes()
    encoded = base64.b64encode(image_bytes).decode("utf-8")
    return f"data:image/png;base64,{encoded}"

async def verify_pdf_page_with_gemma4(
    *,
    task_id: str,
    page_number: int,
    image_path: str,
    expected_metric: str,
    expected_value: str,
) -> dict[str, Any]:
    image_url = image_to_data_url(image_path)

    messages = [
        {
            "role": "system",
            "content": (
                "你是财报视觉复核助手。请只基于图片和给定问题判断，"
                "不要编造图片中不存在的数值。输出 JSON。"
            ),
        },
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": (
                        f"请复核 PDF 第 {page_number} 页是否包含指标："
                        f"{expected_metric} = {expected_value}。"
                        "请返回 found、visual_evidence、confidence、notes。"
                    ),
                },
                {
                    "type": "image_url",
                    "image_url": {"url": image_url},
                },
            ],
        },
    ]

```

7.3 多模态与函数调用结合

更关键的是，多模态不是孤立能力，而是和函数调用结合：

用户问题：

"请核查万华化学 2025 年资产负债率是否来自正确表格。"

Gemma4 执行：

1. 调用 `get_financial_metrics` 查询资产负债率。
2. 调用 `get_source_table` 获取表格索引和 PDF 页码。
3. 调用 `get_pdf_page_image` 获取对应 PDF 页图像。
4. 读取页面图像，检查表格标题、列名、数值位置。
5. 输出带证据的核查结论。

这条链路保证了模型不是"看过上下文后猜测"，而是：

结构化数据 -> 表格来源 -> PDF 页图像 -> 多模态视觉复核 -> 审计结论

8. Gemma4 在工作流中的调用

8.1 工作流入口

工作流 API 位于：

```
apps/api-gateway/routers/workflow.py
```

核心语义增强接口：

```
POST /api/workflow/task/{task_id}/semantic
```

该接口的执行逻辑可以概括为：

1. 检查 PDF 解析产物是否完整
2. 导入 Wiki 报告入口
3. 运行规则语义抽取脚本
4. 读取当前本地模型配置
5. 将 Gemma4 配置注入环境变量
6. 调用 LLM semantic enrichment 脚本
7. 写入 semantic/llm/<report_id>/

8.2 模型环境注入

```
def _llm_semantic_env() -> dict[str, str]:
    env = os.environ.copy()
    settings = load_llm_settings(include_secrets=True)

    local_provider = (settings.get("providers") or {}).get("local") or {}

    if local_provider.get("baseUrl"):
        env["FINSIGHT_LOCAL_LLM_BASE_URL"] = str(local_provider["baseUrl"])
    if local_provider.get("model"):
        env["FINSIGHT_LOCAL_LLM_MODEL"] = str(local_provider["model"])
    if local_provider.get("apiKey"):
        env["FINSIGHT_LOCAL_LLM_API_KEY"] = str(local_provider["apiKey"])
    if local_provider.get("timeoutSeconds"):
        env["FINSIGHT_LLM_SEMANTIC_TIMEOUT"] = str(local_provider["timeoutSeconds"])
    if local_provider.get("maxTokens"):
        env["FINSIGHT_LLM_SEMANTIC_MAX_TOKENS"] = str(local_provider["maxTokens"])

    return env
```

这意味着工作流不会硬编码模型，而是读取设置页保存的当前本地模型。默认情况下，该模型就是 Gemma4。

8.3 LLM 语义增强输出

Gemma4 负责生成：

```
semantic/llm/<report_id>/
  enrichment.json
  business_profile.json
  claims.json
  risks.json
  events.json
  review_queue.json
  extraction_log.json
```

这些文件服务于后续智能体：

表 5 语义增强输出文件说明

文件	作用
business_profile.json	公司业务画像
claims.json	关键经营和财务声明
risks.json	风险因素归纳
events.json	重大事件抽取
review_queue.json	需要人工复核的问题
extraction_log.json	模型调用日志与审计信息

9. Gemma4 在智能体中的作用

9.1 多智能体结构

FinSight 当前包含五类业务智能体：

表 6 FinSight 多智能体结构

智能体	职责
Assistant	财报问答、指标查询、证据溯源
Analysis	生成 14 章年度分析报告
Factchecker	核查报告事实、计算和证据链
Tracking	持续跟踪指标、事项和预警
Legal	法务合规检索和法律意见书

Gemma4 作为本地主模型，可以为这些智能体提供统一推理能力。

9.2 模型切换控制

模型控制逻辑位于：

```
apps/api-gateway/services/hermes_model_control.py
```

当前设计：

```
用户说"切换到本地模型"
-> 默认切到 Gemma4

用户说"切换到 Gemma4"
-> 切到 Gemma4

用户明确说"切换到 Qwen3.6"
-> 切到 Qwen3.6 备用模型

用户说"切回云端"
-> 切回云端 provider
```

本地 fallback 链：

```
1. Gemma-4-26B-A4B-it-NVFP4
2. Qwen3.6-35B-A3B-FP8
```

9.3 智能体调用价值

Gemma4 在不同智能体中的作用如下：

表 7 Gemma4 在各智能体中的作用

智能体	Gemma4 作用
Assistant	根据用户问题选择指标查询、PDF 来源、Wiki 检索工具
Analysis	对结构化证据进行章节化分析，生成经营诊断
Factchecker	对分析报告中的数值、引用、页码和结论进行复核
Tracking	从报告与事件中生成后续跟踪事项和预警规则
Legal	在法规检索结果基础上生成合规判断和法律意见

10. 架构上的安全与可信设计

10.1 不让模型直接相信自己

FinSight 的关键设计原则是：

模型负责推理，证据负责事实。

因此 Gemma4 不能直接编造财务指标，而必须通过工具或结构化证据层获取数据。

10.2 证据优先

所有关键输出都要求绑定：

```
company_id  
report_id  
task_id  
pdf_page_number  
table_index  
source_path  
evidence_id
```

这样可以追溯到：

```
PDF 原文页面  
Markdown 行号  
document_full.json  
financial_data.json  
Wiki evidence  
PostgreSQL 记录
```

10.3 多模态复核降低解析错误

文本解析可能出错，尤其是表格和页码。因此多模态链路用于做第二层复核：

```
Markdown / JSON 结果  
-> 找到 PDF 页  
-> 读取页面图像  
-> Gemma4 视觉复核  
-> 输出 confidence 和 notes
```

11. 与传统 RAG 的区别

FinSight 并不是普通 RAG 系统。

普通 RAG：

```
用户问题 -> 检索片段 -> LLM 生成答案
```

FinSight + Gemma4：

用户问题

- > Gemma4 判断需要哪些工具
- > 调用财务指标 / Wiki / PDF 来源 / 法规检索
- > 必要时读取 PDF 页面图像
- > 生成带证据的回答
- > 输出可审计引用

核心区别：

表 8 普通 RAG 与 FinSight + Gemma4 对比

对比项	普通 RAG	FinSight + Gemma4
数据来源	文本片段	PDF、表格、Wiki、DB、法规库、图像
工具调用	通常没有或弱工具	Native Function Calling
多模态	通常只处理文本	支持 PDF 页面图像复核
审计能力	引用片段	页码、表格索引、证据 ID
输出控制	文本回答	JSON、报告、复核队列、法律意见

12. 参赛亮点总结

12.1 技术亮点

1. Gemma4 本地私有化部署

- 使用 vLLM 暴露 OpenAI-compatible 接口。
- 适合金融和合规场景。

2. Native Function Calling

- Gemma4 可主动调用财务指标、PDF 来源、法规检索等工具。
- 避免模型凭空生成财务事实。

3. Multimodal 财报复核

- 使用 PDF 页面图像进行视觉核查。
- 解决表格错列、页码不一致、扫描版内容等问题。

4. 多智能体协同

- 分析、核查、跟踪、法务各司其职。
- Gemma4 作为本地主模型统一提供推理能力。

5. 证据链可审计

- 输出绑定 PDF 页码、表格索引、Wiki evidence 和数据库记录。

12.2 业务亮点

1. 面向真实 A 股年报研究。
2. 支持从 PDF 到报告的完整链路。
3. 适合投研、审计、合规、企业内控场景。
4. 本地模型降低数据外泄风险。
5. 可作为金融智能体私有化工作台原型。

13. 结论

Gemma4 在 FinSight 中的作用可以概括为：

本地模型底座
+ Native Function Calling 工具推理核心
+ Multimodal 财报视觉复核引擎
+ 多智能体统一本地推理模型

本项目选择 `Gemma-4-26B-A4B-it-NVFP4`，是因为它在模型能力、本地部署成本、函数调用、多模态处理和金融合规场景之间取得了较好的平衡。

通过 Gemma4，FinSight 不只是一个“会生成财报分析”的应用，而是一个能够调用工具、读取证据、复核图像、生成可追溯结论的金融智能体系统。

14. 模型部署指南

本章基于 `start_gemma4_26b_a4b_nvfp4_v11m.sh` 部署脚本，详细介绍如何在本地环境中部署 Gemma-4-26B-A4B-it-NVFP4 模型，以支持 FinSight 系统的本地推理服务。

14.1 部署环境准备

在部署 Gemma4 之前，需要确保系统环境满足以下要求：

表 9 部署环境要求

组件	要求	说明
GPU	NVIDIA GPU (推荐 RTX 4090 / A100 / H100)	NVFP4 量化需要 NVIDIA 硬件支持
CUDA	CUDA 12.x	需与 PyTorch 版本匹配
Python	Python 3.12	脚本依赖 Python 3.12 路径
Conda 环境	vllm-gemma4-nvfp4	预配置 vLLM + NVFP4 依赖
磁盘空间	≥ 100GB	模型文件 + 缓存 + 日志
内存	≥ 64GB RAM	模型加载和推理缓冲
操作系统	Linux (Ubuntu 20.04+)	推荐使用 Linux 服务器环境

14.1.1 Conda 环境配置

部署脚本使用独立的 Conda 环境来隔离 vLLM 及其依赖：

```
# 创建并激活 Conda 环境
conda create -n vllm-gemma4-nvfp4 python=3.12 -y
conda activate vllm-gemma4-nvfp4

# 安装 vLLM 及 Gemma4 支持依赖
pip install vllm --upgrade
pip install transformers accelerate --upgrade
```

14.1.2 CUDA 库路径配置

脚本自动检测以下 CUDA 库路径并配置 `LD_LIBRARY_PATH`：


```
CUDA_LIB_DIRS=(
  "$CONDA_ENV/lib/python3.12/site-packages/torch/lib"
  "$CONDA_ENV/lib/python3.12/site-packages/nvidia/cu13/lib"
  "$CONDA_ENV/lib/python3.12/site-packages/nvidia/cudnn/lib"
  "/usr/local/lib/ollama/cuda_v12"
)
```

环境变量配置：

```
export HF_HOME="/home/maoyd/hf_cache_new"
export VLLM_NVFP4_GEMM_BACKEND="marlin"
export PYTORCH_CUDA_ALLOC_CONF="expandable_segments:True"
```

表 10 关键环境变量说明

环境变量	作用
HF_HOME	Hugging Face 模型缓存目录，避免重复下载
VLLM_NVFP4_GEMM_BACKEND	NVFP4 GEMM 后端实现，使用 marlin 优化
PYTORCH_CUDA_ALLOC_CONF	启用 CUDA 可扩展内存段，减少 OOM 风险

14.2 模型下载与配置

14.2.1 模型来源

本项目使用的模型为 `bg-digitalservices/Gemma-4-26B-A4B-it-NVFP4`，该模型是 Gemma-4-26B-A4B-it 的 NVFP4 量化版本，托管于 Hugging Face Hub。

14.2.2 模型目录结构

```
MODEL_DIR="/home/maoyd/hf_cache_new/hub/\
models--bg-digitalservices--Gemma-4-26B-A4B-it-NVFP4/\
snapshots/a15dd6f161881b62db952303a5bfb7be118ed15e"
```

模型目录可通过以下方式获取：

```
# 方式一：使用 huggingface-cli 下载
huggingface-cli download bg-digital-services/Gemma-4-26B-A4B-it-NVFP4 \
  --local-dir ./Gemma-4-26B-A4B-it-NVFP4

# 方式二：使用 Python 脚本下载
from huggingface_hub import snapshot_download
model_path = snapshot_download(
    repo_id="bg-digital-services/Gemma-4-26B-A4B-it-NVFP4",
    cache_dir="/home/maoyd/hf_cache_new"
)
```

14.2.3 配置参数概览

脚本支持通过环境变量覆盖所有默认参数：

表 11 部署脚本环境变量参数

环境变量	默认值	说明
MODEL_DIR	/home/maoyd/hf_cache_new/hub/.../Gemma-4-26B-A4B-it-NVFP4	模型文件路径
SERVED_MODEL_NAME	Gemma-4-26B-A4B-it-NVFP4	服务暴露的模型名称
HOST	127.0.0.1	服务监听地址
PORT	8006	服务监听端口
MAX_MODEL_LEN	131072	最大上下文长度 (128K)
GPU_MEMORY_UTILIZATION	0.27	GPU 显存利用率
MAX_NUM_BATCHED_TOKENS	4096	最大批处理 token 数
MAX_NUM_SEQS	4	最大并发序列数
DTYPE	bf16	模型权重数据类型
QUANTIZATION	petit_nvfp4	量化方式
MOE_BACKEND	marlin	MoE 后端实现
ENFORCE_EAGER	0	是否强制 eager 模式
LOG_FILE	/home/maoyd/logs/gemma4_26b_a4b_nvfp4_vllm.log	日志文件路径
CONDA_ENV	/home/maoyd/miniconda3/envs/vllm-gemma4-nvfp4	Conda 环境路径
PYTHON_OVERRIDE_DIR	" "	Python 路径覆盖

14.3 vLLM 服务启动

14.3.1 启动命令构建

脚本按以下方式构建 vLLM serve 命令：

```
VLLM_CMD=(  
    "$VLLM_BIN" serve "$MODEL_DIR"  
    --served-model-name "$SERVED_MODEL_NAME"  
    --host "$HOST"  
    --port "$PORT"  
    --max-model-len "$MAX_MODEL_LEN"  
    --gpu-memory-utilization "$GPU_MEMORY_UTILIZATION"  
    --max-num-batched-tokens "$MAX_NUM_BATCHED_TOKENS"  
    --max-num-seqs "$MAX_NUM_SEQS"  
    --dtype "$DTYPE"  
    --quantization "$QUANTIZATION"  
    --trust-remote-code  
    "${VLLM_EXTRA_ARGS[@]}"  
)
```

14.3.2 后台启动

脚本使用 `nohup` + `setsid` 组合确保服务在后台稳定运行：

```
if command -v setsid >/dev/null 2>&1; then  
    nohup setsid "${VLLM_CMD[@]}" >> "$LOG_FILE" 2>&1 &  
else  
    nohup "${VLLM_CMD[@]}" >> "$LOG_FILE" 2>&1 &  
fi  
  
echo "vLLM server started in background, PID: $!"
```

启动流程说明：

1. 创建日志目录（如果不存在）
2. 配置 CUDA 库路径和环境变量
3. 构建 vLLM 命令行参数
4. 以守护进程方式启动服务

5. 输出进程 ID 便于后续管理

14.3.3 完整启动示例

```
# 使用默认配置启动
bash start_gemma4_26b_a4b_nvfp4_vllm.sh

# 使用自定义端口和模型路径启动
PORT=8008 MODEL_DIR=/path/to/model bash start_gemma4_26b_a4b_nvfp4_vllm.sh

# 查看启动日志
tail -f /home/maoyd/logs/gemma4_26b_a4b_nvfp4_vllm.log
```

14.4 关键参数说明

14.4.1 NVFP4 量化参数

表 12 NVFP4 量化关键参数

参数	值	说明
QUANTIZATION	petit_nvfp4	NVIDIA 4-bit Float Point 量化，显著降低显存占用
DTYPE	bfloat16	计算时使用 bfloat16 保持精度
GPU_MEMORY_UTILIZATION	0.27	仅使用 27% GPU 显存，为其他组件留余量

14.4.2 MoE (Mixture of Experts) 参数

表 13 MoE 相关参数

参数	值	说明
MOE_BACKEND	marlin	使用 Marlin 内核加速 MoE 路由计算
总参数	26B	模型总参数量
激活参数	A4B (~4B)	每次前向传播实际激活的专家参数量

14.4.3 性能调优参数

表 14 性能调优参数

参数	值	说明
MAX_MODEL_LEN	131072	支持 128K 上下文，满足长财报分析需求
MAX_NUM_SEQS	4	并发请求数，平衡吞吐与延迟
MAX_NUM_BATCHED_TOKENS	4096	每批次最大 token 数
ENFORCE_EAGER	0 (关闭)	启用 CUDA Graph 加速推理

14.5 服务验证与监控

14.5.1 服务健康检查

启动后，可通过以下方式验证服务是否正常运行：

```
# 检查服务是否监听端口
netstat -tlnp | grep 8006

# 测试模型列表 API
curl http://127.0.0.1:8006/v1/models

# 测试聊天 completions API
curl http://127.0.0.1:8006/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "Gemma-4-26B-A4B-it-NVFP4",
    "messages": [{"role": "user", "content": "Hello"}],
    "max_tokens": 128
  }'
```

14.5.2 监控指标

表 15 服务监控指标

指标	检查方式
服务进程	<code>ps aux grep vllm</code> 或 <code>pgrep -f "vllm serve"</code>
GPU 显存	<code>nvidia-smi</code> 查看显存占用
GPU 利用率	<code>nvidia-smi dmon</code> 实时监测
服务日志	<code>tail -f /home/maoyd/logs/gemma4_26b_a4b_nvfp4_vllm.log</code>
API 响应	<code>curl</code> 测试 chat completions 接口

14.5.3 常见问题排查

表 16 常见问题与解决方案

问题	可能原因	解决方案
CUDA Out of Memory	显存不足或其他进程占用	降低 <code>MAX_MODEL_LEN</code> 或 <code>MAX_NUM_SEQS</code>
模型加载失败	模型文件不完整	重新下载模型，校验文件完整性
端口冲突	8006 端口被占用	修改 <code>PORT</code> 环境变量
CUDA 版本不匹配	PyTorch CUDA 版本与驱动不匹配	检查 <code>nvidia-smi</code> 和 <code>torch.version.cuda</code>
量化加载失败	NVFP4 不支持当前 GPU	确认 GPU 架构支持 NVFP4 (Hopper/Ada Lovelace+)

14.6 与 FinSight 集成

14.6.1 API Gateway 配置

FinSight 的 API Gateway 通过环境变量连接到本地 vLLM 服务：

```
# .env 配置示例
FINSIGHT_LOCAL_LLM_BASE_URL=http://127.0.0.1:8006/v1
FINSIGHT_LOCAL_LLM_MODEL=Gemma-4-26B-A4B-it-NVFP4
FINSIGHT_LOCAL_LLM_API_KEY=
```

14.6.2 服务依赖关系

FinSight 系统启动时，Gemma4 vLLM 服务应最先启动：

1. 启动 Gemma4 vLLM 服务（端口 8006）
`bash start_gemma4_26b_a4b_nvfp4_vllm.sh`
2. 验证服务健康
`curl http://127.0.0.1:8006/v1/models`
3. 启动 FinSight API Gateway
`uvicorn apps.api-gateway.main:app --host 0.0.0.0 --port 8000`
4. 启动 FinSight Web 前端
`cd apps/web && npm run dev`
5. 在设置页验证 Gemma4 连接
访问 `http://localhost:5173/settings` -> 测试本地模型

14.6.3 生产环境建议

表 17 生产环境部署建议

建议	说明
使用 systemd 管理 vLLM 服务	确保服务开机自启和崩溃重启
配置反向代理 (Nginx)	添加 TLS 终止和负载均衡
监控告警	GPU 温度、显存、服务响应时间
日志轮转	避免日志文件无限增长
备份模型文件	防止模型损坏导致服务中断

附录：完整部署脚本

以下为 `start_gemma4_26b_a4b_nvfp4_vllm.sh` 完整内容，可直接用于生产环境部署：

A.1 脚本头部与默认配置

```
#!/usr/bin/env bash
set -euo pipefail

MODEL_DIR="${MODEL_DIR:-/home/maoyd/hf_cache_new/hub/models--bg-digital--Gemma-4-26B-A4B-it-NVFP4/snapshots/a15dd6f161881b62db952303a5bfb7be118ed15e}"
SERVED_MODEL_NAME="${SERVED_MODEL_NAME:-Gemma-4-26B-A4B-it-NVFP4}"
HOST="${HOST:-127.0.0.1}"
PORT="${PORT:-8006}"
MAX_MODEL_LEN="${MAX_MODEL_LEN:-131072}"
GPU_MEMORY_UTILIZATION="${GPU_MEMORY_UTILIZATION:-0.27}"
MAX_NUM_BATCHED_TOKENS="${MAX_NUM_BATCHED_TOKENS:-4096}"
MAX_NUM_SEQS="${MAX_NUM_SEQS:-4}"
DTYPE="${DTYPE:-bfloat16}"
QUANTIZATION="${QUANTIZATION:-petit_nvfp4}"
MOE_BACKEND="${MOE_BACKEND:-marlin}"
ENFORCE_EAGER="${ENFORCE_EAGER:-0}"
LOG_FILE="${LOG_FILE:-/home/maoyd/logs/gemma4_26b_a4b_nvfp4_vllm.log}"
CONDA_ENV="${CONDA_ENV:-/home/maoyd/miniconda3/envs/vllm-gemma4-nvfp4}"
VLLM_BIN="${VLLM_BIN:-$CONDA_ENV/bin/vllm}"
PYTHON_OVERRIDE_DIR="${PYTHON_OVERRIDE_DIR:-}"

mkdir -p "$(dirname "$LOG_FILE")"
```

A.2 环境变量配置

```
export HF_HOME="${HF_HOME:-/home/maoyd/hf_cache_new}"
export VLLM_NVFP4_GEMM_BACKEND="${VLLM_NVFP4_GEMM_BACKEND:-marlin}"
export PYTORCH_CUDA_ALLOC_CONF="${PYTORCH_CUDA_ALLOC_CONF:-expandable_segments:True}"
```

A.3 CUDA 库路径配置

```
CUDA_LIB_DIRS=(
    "$CONDA_ENV/lib/python3.12/site-packages/torch/lib"
    "$CONDA_ENV/lib/python3.12/site-packages/nvidia/cu13/lib"
    "$CONDA_ENV/lib/python3.12/site-packages/nvidia/cudnn/lib"
    "/usr/local/lib/ollama/cuda_v12"
)

VLLM_LD_LIBRARY_PATH=""
for lib_dir in "${CUDA_LIB_DIRS[@]}; do
    if [[ -d "$lib_dir" ]]; then
        if [[ -z "$VLLM_LD_LIBRARY_PATH" ]]; then
            VLLM_LD_LIBRARY_PATH="$lib_dir"
        else
            VLLM_LD_LIBRARY_PATH="${VLLM_LD_LIBRARY_PATH}:$lib_dir"
        fi
    fi
done
export LD_LIBRARY_PATH="${VLLM_LD_LIBRARY_PATH}${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"

if [[ -n "$PYTHON_OVERRIDE_DIR" ]]; then
    export PYTHONPATH="${PYTHON_OVERRIDE_DIR}:${PYTHONPATH:-}"
fi
```

A.4 vLLM 命令构建与启动

```
VLLM_EXTRA_ARGS=(  
  if [[ -n "$MOE_BACKEND" ]]; then  
    VLLM_EXTRA_ARGS+=(--moe-backend "$MOE_BACKEND")  
  fi  
  if [[ "$ENFORCE_EAGER" == "1" || "$ENFORCE_EAGER" == "true" || "$ENFORCE_EAGER" == "TRUE"  
  ]]; then  
    VLLM_EXTRA_ARGS+=(--enforce-eager)  
  fi  
  
  VLLM_CMD=(  
    "$VLLM_BIN" serve "$MODEL_DIR"  
    --served-model-name "$SERVED_MODEL_NAME"  
    --host "$HOST"  
    --port "$PORT"  
    --max-model-len "$MAX_MODEL_LEN"  
    --gpu-memory-utilization "$GPU_MEMORY_UTILIZATION"  
    --max-num-batched-tokens "$MAX_NUM_BATCHED_TOKENS"  
    --max-num-seqs "$MAX_NUM_SEQS"  
    --dtype "$DTYPE"  
    --quantization "$QUANTIZATION"  
    --trust-remote-code  
    "${VLLM_EXTRA_ARGS[@]}"  
  )  
  
  if command -v setsid >/dev/null 2>&1; then  
    nohup setsid "${VLLM_CMD[@]}" >> "$LOG_FILE" 2>&1 &  
  else  
    nohup "${VLLM_CMD[@]}" >> "$LOG_FILE" 2>&1 &  
  fi  
  
  echo "vLLM server started in background, PID: $!"
```